

SRE / DevOps Mastery + APAC SWE Job Search Guide

For: Bangladesh-based Junior SRE Engineer who knows Docker, CI/CD, Kubernetes basics, and a little AWS **Goal:** Master the full software lifecycle (code → deploy → operate → optimize cost → design perfect infrastructure) AND land a visa-sponsored SWE/SRE/DevOps role in Singapore, Japan, Malaysia, or Thailand **Time commitment:** 15 hours/week **Timeline:** 6 months (3 months skill building + 3 months job hunt overlap)

Table of Contents

1. TL;DR — The Big Picture
 2. Part A — Full Software Lifecycle Mastery
 3. Part B — Cloud Deep Dive (AWS Primary)
 4. Part C — Infrastructure as Code, Observability, Cost Optimization
 5. Part D — System Design for SRE/DevOps Interviews
 6. Part E — Coding & Programming for SREs
 7. Part F — 6-Month Week-by-Week Plan
 8. Part G — APAC Job Search (Singapore/Japan/Malaysia/Thailand)
 9. Part H — Interview Preparation Beyond Tech
 10. Part I — Personal Branding & Networking
 11. Part J — Resources Master List
-

TL;DR — The Big Picture

You're in a fantastic position: SRE/DevOps is one of the **highest-paying and most visa-friendly** specializations in APAC. Grab, Shopee, ByteDance, Mercari, PayPay, Rakuten, Agoda — every one of them is hungry for engineers who can run production systems at scale.

Your current state: Junior SRE + Docker + CI/CD + K8s basics + a little AWS = good foundation, but not yet senior-marketable.

What you need to add:

1. **Deep AWS** (Solutions Architect Associate + hands-on with 10–15 core services)
2. **Infrastructure as Code** (Terraform fluency, ideally Pulumi exposure)
3. **Observability stack** (Prometheus, Grafana, Loki, OpenTelemetry, distributed tracing)
4. **Production K8s** (not just `kubectl get pods` — Helm, operators, autoscaling, networking, security)

5. **Cost optimization** (FinOps fundamentals, AWS cost tools, right-sizing)
6. **System design** at SRE depth (capacity planning, SLO/SLI/SLA, reliability patterns)
7. **One strong programming language** beyond shell (Go or Python — Go is the SRE lingua franca)
8. **A public portfolio** — homelab project, blog posts, GitHub repos
9. **Interview prep** — LeetCode (medium-level), behavioral STAR stories, mock interviews

Realistic outcome: With disciplined 15 hrs/week effort, you can be interview-ready for mid-level SRE/DevOps/Platform Engineer roles at APAC companies within **5–6 months**, with offers landing in months 6–8.

Part A — Full Software Lifecycle Mastery

The “full lifecycle” you want to learn breaks down into these stages. For each stage, here’s what mastery looks like and what to learn.

A1. Plan & Design Phase

What it covers: Requirements, architecture decisions, capacity planning, security threat modeling, choosing tech stack.

What to learn:

- **Architecture decision records (ADRs)** — read Michael Nygard’s original 2011 blog post on ADRs. Practice writing 3 ADRs for your current work.
- **C4 model** for documenting architecture (Simon Brown’s c4model.com).
- **Threat modeling basics** — STRIDE framework. Read OWASP Threat Dragon docs.
- **Capacity planning** — back-of-envelope math: requests/sec, storage growth, bandwidth, peak vs avg load. The “Numbers Every Programmer Should Know” classic Jeff Dean list.

Practical exercise: Pick any side project; write a 1-page architecture doc (C4 diagram + ADR for each major tech choice + a back-of-envelope capacity estimate).

A2. Code Phase

What it covers: Writing maintainable code, code reviews, branching strategies, dependency management.

What to learn (SRE-flavored):

- **Trunk-based development vs Git Flow** — most APAC tech companies use trunk-based with feature flags. Read trunkbaseddevelopment.com.
- **Conventional commits** + semantic versioning.
- **Pre-commit hooks** — `pre-commit` framework, `lefthook`. Set up linting + secret scanning (`gitleaks`, `trufflehog`) on every commit.
- **Dependency management** — Dependabot/Renovate, SBOM (Software Bill of Materials), license compliance.
- **Code review culture** — Google’s Engineering Practices documentation (free, online) is the gold standard.

A3. Build & Test Phase

What to learn:

- **CI fundamentals beyond Jenkins:** GitHub Actions deeply, GitLab CI, ArgoCD for GitOps. Understand caching, matrix builds, parallelization.
- **Container best practices:** Multi-stage Dockerfiles, distroless images, Trivy/Grype for image scanning, Cosign for image signing.
- **Testing pyramid:** unit (70%) → integration (20%) → E2E (10%). Learn `testcontainers` for integration tests.
- **Performance testing:** k6, Locust, JMeter. Run a load test against your own service.
- **Security testing:** SAST (SonarQube, Semgrep), DAST (OWASP ZAP), dependency scanning (Snyk, Dependabot).

Hands-on project: Build a “secure CI pipeline” repo that demonstrates all of the above on a simple Go/Python microservice. This becomes a portfolio piece.

A4. Deploy Phase

What to learn:

- **Deployment strategies:** Blue-green, canary, rolling, feature flags. Understand when to use each.
- **GitOps:** ArgoCD or Flux. This is THE deployment pattern for Kubernetes in 2026. Build a demo cluster with ArgoCD managing 3 apps.
- **Progressive delivery:** Argo Rollouts, Flagger. Canary deployments with automated rollback based on Prometheus metrics.
- **Secrets management:** HashiCorp Vault, AWS Secrets Manager, Sealed Secrets, External Secrets Operator. Never put secrets in Git.
- **Service mesh basics:** Istio or Linkerd — what they solve (mTLS, traffic shifting, observability) and when they’re overkill.

A5. Operate Phase

What to learn:

- **The four golden signals** (latency, traffic, errors, saturation) — from the Google SRE book.
- **SLO/SLI/SLA** — define them, measure them, alert on burn rate (not threshold). Read the “Site Reliability Engineering” book chapters 4–6.
- **Incident management:** PagerDuty/Opsgenie, on-call rotations, run-books, post-mortems (blameless culture).
- **Chaos engineering basics:** Chaos Monkey concept, Gremlin, Litmus for K8s. Run one chaos experiment.

A6. Maintain & Improve Phase

What to learn:

- **Toil reduction** — identify repetitive manual tasks, automate them. Track “toil percentage” as an SRE metric.
- **Capacity reviews** — quarterly check on traffic growth vs infra capacity.
- **Technical debt management** — frame as risk + cost, not just “code is ugly.”
- **Postmortem culture** — every incident produces a learning doc. Read public postmortems: GitHub’s status page archives, Cloudflare’s blog, AWS post-event summaries.

A7. Cost Optimization Phase (your specific interest)

What to learn:

- **FinOps Foundation** — free certification “FinOps Certified Practitioner” (~6 hrs of study). The FinOps Framework defines 3 phases: Inform → Optimize → Operate.
- **AWS-specific cost levers:**
 - **Right-sizing:** AWS Compute Optimizer, Trusted Advisor. EC2 instances are usually 30–50% over-provisioned.
 - **Savings Plans + Reserved Instances** — commit to 1–3 years for 30–72% discount.
 - **Spot instances** — up to 90% off for stateless/batch workloads.
 - **Storage tiering** — S3 Intelligent-Tiering, lifecycle policies to Glacier.
 - **Networking** — NAT Gateway costs add up fast; VPC Endpoints save real money.
 - **Idle resources** — unattached EBS volumes, unused Elastic IPs, old snapshots, dev environments running 24/7.
- **Kubernetes cost tools:** OpenCost, Kubecost, Karpenter for autoscaling. Karpenter alone can save 30–50% on EC2 spend for K8s clusters.
- **Tagging strategy** — every resource gets **Owner**, **Environment**, **CostCenter**, **Project** tags. Without tags, cost optimization is guesswork.

Portfolio project: Take a sample AWS account (use the Free Tier), deploy a small app, then write a blog post: “How I reduced my AWS bill from \$X to \$Y.”

Even fictional numbers work if the techniques are real.

Part B — Cloud Deep Dive (AWS Primary)

You said you know “a little bit AWS.” Here’s the path to genuine fluency.

B1. The 15 AWS services you must know deeply

Rank by importance for SRE/DevOps roles:

Service	Why it matters	Time to learn
EC2	Foundation; AMIs, instance families, networking	8 hrs
VPC	Subnets, route tables, NAT, security groups, NACLs, peering	10 hrs
IAM	Roles, policies, trust relationships, STS, OIDC for K8s	8 hrs
S3	Storage classes, lifecycle, versioning, replication, encryption	4 hrs
RDS / Aurora	Multi-AZ, read replicas, backups, parameter groups	6 hrs
EKS	Managed K8s, IRSA, node groups, Fargate	10 hrs
ECS / Fargate	Lighter container option; some companies prefer over EKS	4 hrs
ELB (ALB/NLB)	Load balancing, target groups, listeners, WAF integration	4 hrs
Route 53	DNS, routing policies, health checks	3 hrs
CloudWatch	Metrics, logs, alarms, dashboards, Logs Insights	6 hrs
CloudFormation / CDK	Native IaC (though you’ll mostly use Terraform)	4 hrs
Lambda	Serverless basics, layers, cold starts	4 hrs
SQS / SNS / EventBridge	Messaging, event-driven architecture	4 hrs

Service	Why it matters	Time to learn
Secrets Manager / SSM Parameter Store	Secrets, config	2 hrs
CloudTrail / Config / GuardDuty	Audit, compliance, security	4 hrs

Total: ~80 hrs of focused AWS study = ~5–6 weeks at 15 hrs/week.

B2. AWS Solutions Architect Associate (SAA-C03)

Why: Globally recognized; recruiters in SG/JP filter on it; covers exactly the breadth you need.

Cost: \$150 exam fee. Bangladesh has Pearson VUE test centers in Dhaka (verify current locations on pearsonvue.com/aws).

Best resources (in order):

1. **Stephane Maarek's SAA-C03 course on Udemy** — frequently on sale for \$10–15. THE most recommended course.
2. **Tutorials Dojo practice exams** (Jon Bonso) — \$15–20. If you can score 80%+ on these, you'll pass.
3. **AWS Skill Builder** (free tier) — official AWS training.
4. **Adrian Cantrill's SAA course** (learn.cantrill.io, ~\$50) — deeper than Maarek's, better for genuine understanding (not just exam-passing).

Realistic timeline: 6–8 weeks of focused study → pass on first attempt.

B3. After SAA — what next?

Option 1 (recommended): AWS Certified DevOps Engineer Professional — directly relevant; signals senior-level skills. ~3 months prep after SAA.

Option 2: Certified Kubernetes Administrator (CKA) — \$395, hands-on lab exam. THE credential for K8s roles. Best resource: Mumshad Mannambeth's CKA course on KodeKloud.

Option 3: HashiCorp Terraform Associate — \$70.50, easier than CKA. Quick win + valuable skill.

My recommendation for your trajectory: SAA → CKA → Terraform Associate. By the end you have 3 high-value certs and genuine production-ready skills.

B4. Multi-cloud awareness

Don't go deep on GCP/Azure, but know:

- **GCP** — used heavily by **Mercari**, **LINE**, **Indeed Japan**. Understand GKE, BigQuery, Cloud Run conceptually.
- **Azure** — used by some Singapore banks and Microsoft-aligned companies. Lower priority.

For job interviews, fluency in *one* cloud + conversational familiarity with another is enough.

Part C — Infrastructure as Code, Observability, Cost Optimization

C1. Terraform (mandatory)

Why: It's the industry standard for IaC across AWS, GCP, Azure. Every APAC tech company uses it.

Learning path:

1. **HashiCorp Learn** (learn.hashicorp.com) — free official tutorials. Do the AWS track.
2. **“Terraform Up & Running” by Yevgeniy Brikman** — book, 3rd edition (2022). The single best resource on Terraform at production scale.
3. **Practice projects:**
 - Provision a VPC + EC2 + RDS from scratch.
 - Build a reusable Terraform module (e.g., “secure-s3-bucket” module).
 - Set up Terraform Cloud or use S3 + DynamoDB backend for state.
4. **Advanced topics:** Workspaces, remote state, `terraform import` for legacy infra, Terragrunt for DRY configurations, OpenTofu (the open-source fork) awareness.

Cert: HashiCorp Terraform Associate (\$70.50, 1-hour multiple choice). Worth doing.

C2. Configuration management

Ansible — declining in popularity but still used. Learn enough to recognize playbooks. ~10 hrs total.

Skip Chef and Puppet unless a specific job posting requires them.

C3. Observability Stack (the modern SRE essential)

This is where Junior SREs become Senior SREs. **Mastering observability is your #1 differentiator.**

The three pillars:

1. **Metrics** — Prometheus + Grafana. Time-series data. “Is the system healthy?”
2. **Logs** — Loki / ELK (Elasticsearch + Logstash + Kibana) / OpenSearch / Splunk. “What happened?”
3. **Traces** — Jaeger / Tempo / OpenTelemetry. “Where in the request flow did it slow down?”

Learn in this order:

- **Prometheus fundamentals:** scrape configs, PromQL queries, recording rules, alerting rules. Free course: “Prometheus Certified Associate” study materials.
- **Grafana:** dashboards, alerting, datasource configuration. Build 3 production-quality dashboards.
- **OpenTelemetry (OTel):** the emerging standard. Instrument a sample app with OTel SDK; export to Jaeger.
- **Distributed tracing concepts:** spans, context propagation, sampling strategies.
- **Log aggregation:** Loki for K8s, ELK for traditional, CloudWatch Logs for AWS-native.

Hands-on project: Build a “demo-microservices” repo with 3 services, instrument them with OTel, deploy to a K8s cluster (kind/minikube locally is fine), and set up full Prometheus + Grafana + Loki + Tempo observability. Blog about it. **This single project will impress every SRE interviewer.**

C4. SRE-specific concepts to master

- **SLI/SLO/SLA** — Service Level Indicator (a measurement), Objective (your target), Agreement (legal commitment). Understand the math.
- **Error budgets** — “we allow X minutes of downtime per quarter; once exhausted, freeze feature work and prioritize reliability.”
- **Burn rate alerts** — multi-window, multi-burn-rate alerting (Google’s recommended approach, much better than threshold-based).
- **Runbooks** — every alert should link to a runbook. Practice writing one for a real incident.
- **Postmortems** — blameless format. Read 5 public postmortems from Cloudflare, GitHub, AWS to internalize the structure.

Must-read books:

1. **“Site Reliability Engineering”** (Google, free at sre.google/books) — the original SRE bible. Read chapters 1–6, 11–18.
2. **“The Site Reliability Workbook”** (Google, free) — practical companion.
3. **“Seeking SRE”** (David Blank-Edelman, ed.) — diverse SRE perspectives.

C5. Cost Optimization Deep Dive (your specific interest)

FinOps Foundation resources (mostly free):

- **FinOps Framework** at finops.org — read the principles, phases, and personas.
- **FinOps Certified Practitioner** — \$325 cert; ~6 hrs of study. Marketable in cloud-heavy companies.

Practical AWS cost tools (build hands-on familiarity):

- **AWS Cost Explorer** — daily/monthly cost trends; group by service, account, tag.
- **AWS Budgets** — alerts when forecasted spend exceeds threshold.
- **AWS Compute Optimizer** — right-sizing recommendations for EC2, EBS, Lambda.
- **AWS Trusted Advisor** — security + cost + performance recommendations.
- **AWS Cost Anomaly Detection** — ML-based unusual spend alerts.

Third-party tools (mention in interviews):

- **Vantage, Cloudability, CloudHealth** — multi-cloud FinOps platforms.
- **Kubecost / OpenCost** — K8s-specific cost allocation.
- **Spot.io (formerly Spotinst)** — automated spot instance management.

Cost optimization checklist (memorize this for interviews):

1. Right-size EC2 instances (use Compute Optimizer).
2. Purchase Savings Plans / RIs for steady-state workloads.
3. Use Spot for fault-tolerant workloads.
4. Delete unattached EBS volumes, old snapshots, unused Elastic IPs.
5. Implement S3 lifecycle policies (move cold data to Glacier).
6. Use VPC Endpoints to avoid NAT Gateway data charges.
7. Schedule non-prod environments to stop nights/weekends (~65% saving).
8. Enable Auto Scaling with predictive scaling for variable workloads.
9. Use Karpenter for K8s node autoscaling (vs Cluster Autoscaler).
10. Audit data transfer costs — cross-AZ and cross-region transfers add up.

C6. Infrastructure design patterns to understand

- **The Twelve-Factor App** (12factor.net) — mandatory reading for cloud-native design.
- **The Well-Architected Framework** (AWS) — 6 pillars: Operational Excellence, Security, Reliability, Performance, Cost Optimization, Sustainability. Read at least the Reliability + Cost pillars in full.
- **CNCF Cloud Native Trail Map** (cncf.io) — visual roadmap of the cloud-native ecosystem.

- **Microservices patterns** — Chris Richardson’s microservices.io. Patterns: API Gateway, Service Discovery, Circuit Breaker, Saga, CQRS, Event Sourcing.
-

Part D — System Design for SRE/DevOps Interviews

System design questions for SRE/DevOps roles are slightly different from pure SWE. They focus more on:

- Reliability, scalability, observability
- Deployment architecture (how do you ship + rollback safely?)
- Failure modes (what breaks when AZ X goes down?)
- Capacity planning (how many servers for X requests/sec?)
- Cost-aware design

D1. Core resources

1. **“Designing Data-Intensive Applications” by Martin Kleppmann** — THE book. Read all 12 chapters. Take notes. This is non-negotiable for serious system design.
2. **Alex Xu** — **“System Design Interview Vol 1 & 2”** — interview-focused, more concise.
3. **ByteByteGo** (newsletter + paid platform) — Alex Xu’s daily content.
4. **“Understanding Distributed Systems” by Roberto Vitillo** — excellent middle ground between Kleppmann’s depth and Xu’s brevity.

D2. SRE-specific system design topics

- Designing for **multi-region, multi-AZ** high availability.
- **Database scaling** — read replicas, sharding, CDC (Change Data Capture).
- **Caching strategies** — cache-aside, read-through, write-through, write-behind. Redis vs Memcached.
- **Message queues** — Kafka vs RabbitMQ vs SQS. When to use which.
- **Rate limiting & throttling** — token bucket, leaky bucket algorithms.
- **Circuit breakers & bulkheads** — Hystrix patterns (even though Hystrix is deprecated, the concepts are universal).
- **Idempotency** in distributed systems.
- **Consistency models** — strong, eventual, causal, read-your-writes.

D3. Practice format

For every system design question:

1. **Clarify requirements** (functional + non-functional) — 5 min.
2. **Back-of-envelope math** — QPS, storage, bandwidth — 3 min.

3. **High-level architecture** — boxes and arrows — 8 min.
4. **Deep dive into 2–3 components** — 15 min.
5. **Discuss bottlenecks + scaling** — 10 min.
6. **Discuss reliability + observability + cost** — 5 min (where you shine as an SRE!).

D4. SRE flavor questions to practice

- Design a CI/CD pipeline for 200 microservices.
- Design monitoring for a system handling 1M req/sec.
- Design a blue-green deployment system across 3 regions.
- Design a log aggregation pipeline for 10TB/day.
- Design a multi-region active-active database setup.
- Design a chaos engineering platform.
- Design a feature flag service.
- Design a secrets management system from scratch.

D5. Mock interview platforms

- **Pramp** (free) — peer-to-peer.
- **interviewing.io** (\$200–400/session) — anonymous mocks with FAANG engineers. The single best investment if you can afford 2–3 sessions.
- **Hello Interview** (hellointerview.com) — newer; good walkthrough content.
- **Exponent** — behavioral + system design subscription.

Part E — Coding & Programming for SREs

SRE/DevOps interviews have lighter coding bars than pure SWE, but you still need to clear them.

E1. Pick ONE primary language

Go is the SRE/DevOps lingua franca:

- Kubernetes, Docker, Terraform, Prometheus, etcd, Helm — all written in Go.
- Grab, Mercari, PayPay, ByteDance use Go heavily.
- Easy to learn (small syntax surface), fast, statically typed.

Python is the practical alternative:

- Better for scripting, automation, AWS Lambda.
- Universally useful.

My recommendation: Learn Go as your primary, keep Python as your scripting language.

Go learning path (~40 hrs):

1. **A Tour of Go** (tour.golang.org) — official interactive intro. ~5 hrs.
2. **“Learning Go” by Jon Bodner** (O’Reilly) — best intermediate book.
3. **Go by Example** (gobyexample.com) — pattern reference.
4. Build a small CLI tool in Go (e.g., a `kubectl` plugin, a log parser).
5. Solve 30 LeetCode problems in Go to build fluency.

E2. LeetCode strategy for SRE/DevOps roles

You don’t need to grind 500 problems. Aim for **150 problems** with the right mix:

- **NeetCode 150** — free at neetcode.io. THE recommended list for time-constrained prep.
- Focus on: Arrays, Strings, Hash Maps, Two Pointers, Sliding Window, Binary Search, Trees (BFS/DFS), Heaps, Graphs (basic).
- **Skip the hardest topics** (DP-hard, advanced graph algorithms) unless you have time.

Time allocation: 1 hour/day, 5 days/week $\times$ 12 weeks = ~60 hrs of LeetCode $\rightarrow$ enough for SRE bars.

Interview-specific tips:

- Talk through your approach before coding.
- Start with a brute force, then optimize.
- Test with examples, including edge cases.
- Most SRE coding rounds are EASY + MEDIUM, not HARD.

E3. Bash/Shell scripting

Non-negotiable for SRE. You should be able to:

- Write a shell script that parses logs and extracts patterns.
- Use `awk`, `sed`, `grep`, `jq`, `yq` fluently.
- Write a `Makefile` that orchestrates a project’s common tasks.

Resource: “The Linux Command Line” by William Shotts (free at linuxcommand.org).

E4. SQL fluency

Many interviews include 1–2 SQL questions. You should know:

- JOINS (INNER, LEFT, RIGHT, FULL OUTER).
- GROUP BY, HAVING, window functions (`ROW_NUMBER`, `RANK`, `LAG`, `LEAD`).
- Query optimization basics — EXPLAIN, indexes, N+1 query problem.

Practice: LeetCode SQL section (Top 50 SQL questions).

Part F — 6-Month Week-by-Week Plan

Weekly time budget: 15 hours

- 5 hrs technical study (cloud + IaC + observability)
- 4 hrs hands-on projects
- 3 hrs LeetCode + system design
- 2 hrs job hunt activities (CV, LinkedIn, applications)
- 1 hr reading (books, blogs, postmortems)

Month 1 — AWS Foundations + Go Basics

Week 1:

- Start Stephane Maarek’s SAA-C03 course (sections 1–4: IAM, EC2, EBS).
- Start “A Tour of Go.”
- Set up personal AWS account (Free Tier); enable billing alerts.
- Update LinkedIn headline: “SRE Engineer | AWS | Kubernetes | Open to Singapore/Japan/Malaysia/Thailand opportunities.”

Week 2:

- SAA course sections 5–8: ELB, ASG, EBS deep, EFS.
- Finish Tour of Go; start “Learning Go” book Ch. 1–4.
- Side project: provision a VPC + EC2 web server manually in AWS console.
- Begin daily LeetCode (1 problem/day from NeetCode 150).

Week 3:

- SAA course sections 9–12: S3, CloudFront, Storage Gateway.
- Continue “Learning Go” Ch. 5–8.
- Side project: write a Bash script that audits your AWS account for unattached EBS volumes and unused Elastic IPs.
- LeetCode: 7 more problems.

Week 4:

- SAA course sections 13–17: IAM advanced, VPC deep, NAT, peering.
- Side project: deploy a simple “hello world” Go app to EC2 with proper IAM role + S3 logging.
- Write first blog post: “What I learned in my first month of serious AWS study.”
- LeetCode: 7 more problems (running total: 14).

Month 1 deliverables: SAA course 50% complete; basic Go fluency; 1 blog post; 14 LeetCode problems; LinkedIn optimized.

Month 2 — AWS Mastery + Terraform Start

Week 5:

- SAA course sections 18–21: RDS, Aurora, ElastiCache, DynamoDB.
- Start HashiCorp Learn Terraform AWS track.
- Side project: rewrite your week 4 EC2 deployment in Terraform.
- LeetCode: 7 problems.

Week 6:

- SAA course sections 22–25: Route 53, CloudFront, API Gateway.
- “Terraform Up & Running” book Ch. 1–4.
- Side project: build a reusable Terraform module for a secure S3 bucket (versioning, encryption, lifecycle policy, blocked public access).
- LeetCode: 7 problems.

Week 7:

- SAA course remaining sections: CloudWatch, X-Ray, Lambda, ECS, SQS, SNS.
- “Terraform Up & Running” Ch. 5–8.
- Side project: provision EKS cluster using Terraform; deploy a sample app.
- LeetCode: 7 problems.

Week 8:

- **Take SAA-C03 exam.** Do Tutorials Dojo practice exams until consistently scoring 80%+.
- Side project: write blog post #2 documenting your full Terraform → EKS deployment.
- LeetCode: 7 problems (running total: 42).

Month 2 deliverables: AWS SAA passed; Terraform fluent; EKS cluster from scratch; 2 blog posts; 42 LeetCode problems.

Month 3 — Kubernetes Deep + Observability

Week 9:

- Start CKA course (Mumshad Mannambeth on KodeKloud).
- Deploy Prometheus + Grafana on your EKS cluster.
- LeetCode: 7 problems; start system design study with Alex Xu Vol 1 Ch. 1–4.

Week 10:

- CKA: pods, deployments, services, ingress, ConfigMaps, Secrets.
- Set up alerting rules in Prometheus; create 2 Grafana dashboards.
- Read DDIA Ch. 1–3.
- LeetCode: 7 problems.

Week 11:

- CKA: networking deep, NetworkPolicies, storage, StatefulSets.
- Add Loki + Promtail for log aggregation.

- DDIA Ch. 4–6.
- LeetCode: 7 problems; 1 system design problem.

Week 12:

- CKA: RBAC, security contexts, troubleshooting, etcd backup/restore.
- Instrument a sample app with OpenTelemetry; export traces to Tempo or Jaeger.
- DDIA Ch. 7–9.
- LeetCode: 7 problems (running total: 70).
- Blog post #3: “Building a full observability stack with Prometheus, Grafana, Loki, and Tempo.”

Month 3 deliverables: CKA course 80% complete; full observability stack running; 3 blog posts; 70 LeetCode problems.

Month 4 — CKA Exam + Job Hunt Begins

Week 13:

- **Take CKA exam.** Practice `killer.sh` simulations.
- Update CV with new certs + portfolio links.
- Start applying: 5 applications/week to Grab, Shopee, ByteDance, Mercari, PayPay, Agoda.
- LeetCode: 7 problems.

Week 14:

- Start “Terraform Associate” study (~20 hrs total).
- Mock system design interview on Pramp.
- DDIA Ch. 10–12.
- 5 more applications.

Week 15:

- Take Terraform Associate exam.
- Mock LeetCode + behavioral on Pramp.
- Read 5 public postmortems; write a fake postmortem for a fictional incident as a writing sample.
- 5 more applications.

Week 16:

- Refine application strategy based on response rate.
- First real recruiter screens likely happening.
- Continue LeetCode (target: 100 total by end of Month 4).
- Blog post #4: “From Junior to mid-level SRE: 4-month transformation.”

Month 4 deliverables: CKA + Terraform Associate certs; 20 applications submitted; 100 LeetCode problems; 4 blog posts.

Month 5 — Interview Loops

Week 17–20:

- 8–15 interview rounds expected this month.
- Continue mock interviews weekly (Pramp + 1 paid interviewing.io session).
- Maintain LeetCode at 5 problems/week to stay sharp.
- Re-read DDIA chapters relevant to upcoming interviews.
- Network actively on LinkedIn — connect with 5 engineers/week at target companies.

Month 6 — Offer Evaluation + Negotiation

Week 21–24:

- Convert interview pipeline to offers.
 - Negotiate (more on this below).
 - Visa paperwork begins for chosen offer.
 - Transition planning at Cefalo.
-

Part G — APAC Job Search (Singapore / Japan / Malaysia / Thailand)

G1. Why these countries are great for Bangladeshi SREs

- **High demand:** Every major APAC tech company runs production K8s + AWS/GCP at scale. Senior SRE/DevOps engineers are perpetually short-staffed.
- **Visa-friendly:** All four countries have streamlined work visa pathways for tech professionals.
- **Compensation:** SG and JP especially offer salaries 4–8× what you’d earn in Bangladesh.
- **Strong Bangladeshi diaspora:** Singapore in particular has thousands of Bangladeshi engineers — referrals are real.

G2. Country-by-country breakdown

Singapore (Highest priority for SRE roles) **Why:** Singapore is the APAC headquarters for most regional tech companies. SRE/DevOps roles are abundant.

Top employers actively hiring + sponsoring:

Company	Stack	Notes
Grab	Go, AWS, K8s, Kafka	Largest tech employer; SG-headquartered. Strong SRE org.
Shopee / Sea Group	Go, Java, AWS, GCP	Heavy K8s usage; high-volume hiring.
ByteDance / TikTok SG	Go, C++, K8s	Massive infra; competitive comp.
Stripe APAC	Ruby, Go, AWS	Premium comp; very high bar.
Visa, Mastercard, JPMorgan APAC	Java, AWS	Strong stability + benefits.
GovTech Singapore	Cloud-native, AWS	Hires foreigners for specialist roles.
DBS, OCBC, UOB	Java, K8s, AWS/Azure	Banks heavily investing in cloud-native.
Razer, Ninja Van, Carousell	Mixed	Mid-tier; visa-friendly.
Indeed Singapore	AWS, K8s	Strong English-only environment.
Snowflake, Databricks, ServiceNow	Cloud-native	Premium comp.

Visa: Employment Pass (EP)

- Minimum monthly salary (as of Jan 2025): **SGD 5,600** (general), **SGD 6,200** (financial sector).
- From **Jan 1, 2027**: rising to SGD 6,000 (general) / SGD 6,600 (financial).
- **COMPASS scoring**: 40 points needed. SWE/SRE roles get +20 from Shortage Occupation List bonus.
- Software engineering is on the Shortage Occupation List as of 2024.
- Processing: 3 weeks online (8 weeks if employer is small or overseas).
- Employer applies on your behalf.

Salary expectations (3 YOE SRE):

- Junior SRE: SGD 6,000–8,500/month base.
- Mid SRE: SGD 8,500–13,000/month base.
- Total comp (with bonus + stock): SGD 90K–180K/year for mid-level at Grab/Shopee/ByteDance.

Cost of living: ~SGD 3,000–4,500/month for comfortable single living (Bangladeshi expat). You'll save SGD 4,000–7,000/month easily on a mid-SRE salary.

Best job channels (priority order):

1. **Referrals** — connect with Bangladeshi engineers at Grab/Shopee on LinkedIn; many will refer if profile is solid.
2. **LinkedIn direct apply** — recruiters in SG search actively.
3. **NodeFlair** (nodeflair.com) — Singapore-specific tech jobs with salary data.
4. **MyCareersFuture** (govt-mandated postings; required for EP).
5. **Glints** — large Southeast Asia jobs platform.

Tax: Progressive, 0–22%. At SGD 100K/year, effective rate ~7–9%.

Japan (Best for stable long-term + culture) Why: Japan has a massive engineer shortage. Many companies actively recruit international SREs. Salaries are lower than SG in absolute terms but cost of living is comparable, and the career stability is exceptional.

Top English-friendly employers (verified):

Company	Stack	Japanese needed?
Mercari / Merpay PayPay	Go, GCP, K8s	NO — English-first eng teams
Rakuten	Kotlin, AWS, K8s	Either English OR Japanese (business-level in one)
LINE / LY Corporation	Java, Go, OpenStack	English fluency required (Englishnization since 2010)
Indeed Japan	Java, Go, K8s	Japanese preferred but English-friendly subteams exist
Woven by Toyota	Mixed, AWS	NO — English-first
Money Forward, SmartNews	C++, ROS, Python	NO — English-first
HENNGE, Treasure Data	Mixed	Mostly English
DeNA, Cookpad, Sansan	Mixed	English-friendly
		Japanese helpful

Visa options:

1. **Engineer / Specialist in Humanities visa** (most common):
 - Requires bachelor's degree OR 10+ years experience matching the role.
 - Salary ~¥3M+/year typical minimum.
 - Valid 1, 3, or 5 years; renewable.
2. **Highly Skilled Professional (HSP) visa** (faster, more privileges):
 - Points-based system (70 points required).

- For a 27-year-old with BS + 3 yrs experience + ¥7M+ salary + IELTS 7+ → typically scores 70–80 points.
 - Benefits: 5-year initial visa, fast-track to permanent residency (3 years if 70+ pts, 1 year if 80+ pts), spouse can work, easier to bring parents.
 - **Strongly recommended over Engineer visa.**
3. **J-Skip** (super-fast track):
- Requires MS/PhD + ¥20M+ salary OR 10+ years exp + ¥20M+ salary.
 - Not relevant yet for you.

Salary expectations (3 YOE SRE in Tokyo):

- PayPay: ¥6M–10M/year total comp.
- Mercari: ¥7M–12M.
- Rakuten: ¥6M–9M.
- Indeed Japan, Money Forward: ¥7M–11M.
- (USD equivalent: ~\$45K–80K. Lower than SG in USD but JP cost of living is also lower and quality of life higher.)

Cost of living Tokyo:

- Rent: ¥80K–180K/month (1R/1K apartment).
- Total monthly expenses: ¥200K–280K for comfortable single living.
- You'll save ¥250K–500K/month on a ¥8M+ salary.

Best job channels:

1. **TokyoDev** (tokyodev.com) — THE platform for English-speaking engineering jobs in Japan. Curated, often includes salary.
2. **Japan Dev** (japan-dev.com) — similar, also excellent.
3. **LinkedIn** — recruiters from Robert Walters, Wahl+Case, Computer Futures, Michael Page Japan.
4. **Wantedly** — Japanese-style “casual coffee chat before formal interview” platform.
5. **Bizreach** — mid-senior level focus.
6. **Direct company sites** (Mercari, PayPay, Rakuten careers).

Japanese language: should you learn it?

- **Not required** at PayPay, Mercari, Indeed Japan, Woven, HENNGE for engineering roles.
- **Strongly helpful** for daily life, even basic JLPT N5 (~100 hrs of study via Tofugu/WaniKani/Genki textbook).
- **Required** at most traditional Japanese companies (LINE Japan, DeNA Japanese teams, banks).
- **My recommendation:** Start N5 prep in parallel; doesn't slow your job hunt but improves quality of life dramatically.

Tax in Japan: ~25-30% effective at ¥10M/year (income tax + social insurance).

Malaysia (Cheap, easy, but tightening) **Why:** Lower bar than SG/JP, KL has a growing tech scene, cost of living is very low.

WARNING: Employment Pass salary thresholds **rising effective 1 June 2026:**

- Cat I: RM20,000/month (highest tier, easiest renewal).
- Cat II: RM10,000–19,999/month (most realistic for 3-YOE SRE).
- Cat III: RM5,000–9,999/month (limited renewal).

Apply before June 2026 if possible to potentially get evaluated under old rules.

Top employers:

- **Shopee Malaysia, GrabKL, Lazada KL** — APAC tech hubs.
- **AirAsia / Capital A** — heavy cloud transformation.
- **ByteDance KL** — Southeast Asia operations.
- **Maybank, CIMB** — Malaysian banks investing in tech.
- **IBM Cyberjaya, Intel Penang** — multinational tech.
- **Razer Malaysia** — gaming hardware + cloud.

Salary (3 YOE SRE): RM10,000–18,000/month base = RM120K–220K/year (~USD 25K–47K).

Cost of living KL: Very low. RM5,000–7,000/month covers comfortable single living. High savings rate possible.

Best channels:

1. **JobStreet** (jobstreet.com.my) — dominant local platform.
2. **LinkedIn**.
3. **Hiredly** (formerly WOBB).

Thailand (Niche, Agoda-focused) **Why:** Bangkok is a beautiful place to live, low cost of living, but tech sector is smaller. Mainly relevant if Agoda is your target.

Top employers:

- **Agoda Bangkok** — by far the biggest hirer of foreign SREs. Stack: Scala, Kotlin, MS SQL, Couchbase, AWS. Sponsors family visas.
- **LINE MAN Wongnai** — food delivery tech.
- **SCB 10X, KBTG** — Thai bank tech innovation arms.
- **Tencent Thailand, Sea Thailand** — regional ops.

Visa: Non-B + Work Permit

- Requires bachelor's degree.

- Employer-sponsored.
- Processing 5–7 days for visa + ~10 days for work permit after arrival.
- The **SMART Visa was restricted in Feb 2025** to startup founders only (SMART-S category); employee SWEs use Non-B.
- **LTR Visa** alternative: 10-year residence + 17% flat tax for high-earners (USD 80K+ salary at qualifying employer). Pursue this 1–2 years into your role.

Salary at Agoda (3 YOE): THB 1.7M–2.5M/year (~USD 47K–70K). Senior packages reach THB 4M+.

Cost of living Bangkok: Very low. THB 50K–80K/month for comfortable living. Strong savings potential.

G3. Application strategy (all 4 countries)

Channel mix by country:

Country	Best Channel	Secondary	Tertiary
Singapore	Referrals	LinkedIn direct	NodeFlair
Japan	TokyoDev/Japan Dev	LinkedIn (recruiters)	Direct apply
Malaysia	JobStreet + direct	LinkedIn	Referrals
Thailand	Direct (Agoda site)	LinkedIn	Referrals

Referral hunting playbook:

1. Search LinkedIn: "Bangladesh" AND "Grab" (or Shopee, Mercari, etc.).
2. Look for engineers with shared background (NSU, BUET, IUT, Cefalo, etc.).
3. Send a thoughtful connection message:

“Hi [Name], I’m a fellow Bangladeshi SRE currently at Cefalo. I’m exploring opportunities at [Company] and would love to hear about your experience there. No expectations of a referral — just genuinely curious about the engineering culture. Open to a quick 15-min chat if you have time?”
4. After 2–3 messages of real conversation, *if* fit feels good, ask: “If you think my profile is a fit for the [specific role], I’d be grateful for a referral. Happy to share my CV.”

Referral conversion rates: Per industry studies, referred candidates are roughly 4× more likely to receive offers than direct applicants, and make up ~7% of applications but ~40% of hires.

G4. CV/Resume optimization for APAC

Format:

- 1 page (junior–mid level) or 2 pages MAX (senior).
- PDF only.
- Clean ATS-friendly layout (no fancy graphics, columns, or photos).
- Section order: Header → Summary (2 lines) → Skills → Experience → Projects → Education → Certifications.

Header must include:

- Name + phone (with country code) + email.
- LinkedIn URL (custom, e.g., linkedin.com/in/yourname).
- GitHub URL.
- Personal blog / portfolio URL.
- Location: “Dhaka, Bangladesh — Open to relocation, visa sponsorship required.”

Skills section (keyword-dense, ATS-optimized):

Cloud: AWS (EC2, VPC, IAM, S3, EKS, RDS, Lambda, CloudFront, Route 53), GCP (basic)
 Containers & Orchestration: Docker, Kubernetes, Helm, ArgoCD
 IaC: Terraform, CloudFormation, Ansible
 CI/CD: GitHub Actions, GitLab CI, Jenkins, ArgoCD
 Observability: Prometheus, Grafana, Loki, OpenTelemetry, Jaeger, CloudWatch
 Languages: Go, Python, Bash, SQL
 Databases: PostgreSQL, MySQL, Redis, DynamoDB
 Other: Linux, Git, Nginx, Kafka (basic), service mesh concepts
 Certifications: AWS SAA, CKA, Terraform Associate

Experience bullets — use STAR + quantify:

- “Improved deployment pipeline”
- “Reduced deployment time from 25 minutes to 4 minutes by migrating Jenkins to GitHub Actions with parallel job execution, enabling 3× more daily releases across 12 services”

Every bullet should answer: What did you do? How did you do it? What was the measurable impact?

Projects section (critical for junior–mid candidates):

- Each project: name + 1-line description + tech stack + link to GitHub + 2–3 bullets of impact.
- Aim for 3–4 strong projects.

G5. LinkedIn optimization

Headline: Pack with keywords + intent.

“Site Reliability Engineer | AWS, Kubernetes, Terraform | Open to SRE/DevOps roles in Singapore, Tokyo, KL, Bangkok | Visa sponsorship required”

About section (3–4 short paragraphs):

1. Who you are (years of experience, focus areas).
2. Top achievements (1–2 quantified wins).
3. What you’re looking for (location + role type).
4. CTA (open to messages).

Featured section:

- Pin your best blog post.
- Pin your strongest GitHub project.
- Pin any speaking/community contributions.

Skills section:

- List 15–25 relevant skills.
- Profiles with 15+ endorsed skills receive significantly more recruiter views than those with fewer than 5.
- Ask 5 colleagues at Cefalo to endorse your top 5 skills.

Open to Work setting:

- Turn on “Recruiters only” (so it doesn’t show publicly).
- Add all 4 target cities + remote.

Content strategy:

- Post 1–2× per week: project updates, learning notes, postmortem analyses, hot takes on cloud-native trends.
- Comment thoughtfully on 5 posts/week by APAC recruiters and engineering leaders.
- 3 months of consistent posting will dramatically increase recruiter inbound.

Part H — Interview Preparation Beyond Tech

H1. The full interview loop structure

Typical SRE/DevOps interview loop (5 rounds):

1. **Recruiter screen** (30 min) — basic background, salary expectations, visa status.
2. **Hiring manager call** (45 min) — career narrative, role fit, technical context check.
3. **Coding round** (60 min) — 1–2 LeetCode mediums OR a debugging exercise.
4. **System design round** (60 min) — design a scalable/reliable system.
5. **Behavioral / culture fit** (45 min) — STAR questions, values alignment.

Some companies add:

- **Take-home challenge** (PayPay specifically; 3–4 hours).
- **Live troubleshooting** (debug a broken K8s deployment, fix a Terraform error).
- **Bar raiser** (Agoda, Shopee) — cross-functional senior evaluator.

H2. STAR behavioral framework

Every story you tell should follow:

- **Situation** — context (1–2 sentences).
- **Task** — what was your responsibility (1 sentence).
- **Action** — what *you* specifically did (3–5 sentences). Use “I”, not “we”.
- **Result** — measurable outcome + what you learned (1–2 sentences).

H3. Stories you must have ready (write them out, memorize the structure)

Prepare written 200-word STAR stories for each:

1. **A production incident you handled.** What broke, how you diagnosed it, how you fixed it, what you changed to prevent recurrence.
2. **A time you disagreed with a senior engineer.** How you raised the concern, the outcome, what you learned.
3. **A time you took initiative without being asked.** Spotted a problem, fixed it, owned the result.
4. **A failure or mistake.** Be honest, focus on learning + behavior change.
5. **A time you mentored or helped a teammate.** Demonstrates collaboration.
6. **A technical decision you made and its tradeoffs.** Shows you think holistically.
7. **A time you improved a process or system.** Quantify the impact.
8. **A time you missed a deadline.** Honest framing + how you communicated + lessons.

H4. Reverse questions (you ask THEM)

Always have 5+ thoughtful questions ready. Examples:

- “How does the team measure success for SREs here? SLO-driven, ticket throughput, project delivery, or a mix?”
- “What does on-call look like in practice — frequency, escalation paths, post-incident process?”
- “How does the team balance reliability investments against feature delivery pressure?”
- “What’s the team’s relationship with the product engineering teams it supports?”
- “If I joined, what would success in the first 90 days look like?”

- “What’s the most challenging part of working here that you don’t see mentioned in the JD?”

Avoid: Salary, vacation, working hours in early rounds (save for offer-stage discussions).

H5. Cultural calibration by country

Singapore:

- Direct, business-like, English-fluent.
- No special cultural prep needed beyond standard professionalism.
- They value clarity, efficiency, results.

Japan (especially Mercari, PayPay, Rakuten):

- **Read company values pages before each interview** (Mercari’s “Go Bold, All for One, Be a Pro”; PayPay’s “Day 1 mindset”).
- Frame stories around **team consensus** rather than lone-hero narratives.
- Be respectful, slightly more formal.
- Be ready for “Why Japan?” — have a real, considered answer (don’t say “because anime”).
- Salary negotiation: defer until formal offer stage; aggressive early negotiation is off-putting.

Malaysia:

- Multicultural, English-fluent, business norms similar to SG.
- Slightly more relationship-driven than SG.

Thailand:

- More relaxed, but still professional.
- Agoda specifically is very international; behaves like a global tech company.

H6. Salary negotiation playbook

Phase 1: Before interviewing

- Research using **levels.fyi** (best tool), **Glassdoor**, **NodeFlair** (SG), TokyoDev (JP).
- Have a target range AND a walk-away minimum.

Phase 2: Recruiter screen

- If asked “What’s your expected salary?” → respond: “I’m targeting a competitive market rate for the role and would love to understand the band you have for this position first.”
- If pushed: give a *range* (your target $\pm 15\%$), positioned high but defensible.

Phase 3: After offer

- Always thank them, then ask for 2–5 days to review.
- Negotiate in writing (email).
- Negotiate the *total package*: base, bonus, stock/RSU, sign-on, relocation, visa support, vacation.
- A simple ask: “I’m very excited about this opportunity. Based on my research and the other conversations I’m having, I was hoping the base could be at SGD X. Is there flexibility?”
- If they can’t move base, ask about sign-on, RSU, or relocation budget.

Realistic uplift: 5–15% on base is achievable in most APAC offers if you have another offer or strong leverage.

Part I — Personal Branding & Networking

I1. Build a personal site

- **Domain:** yourname.dev or yourname.com (~\$15/year).
- **Tech:** Hugo, Astro, or Next.js + Vercel/Netlify (free hosting).
- **Pages:**
 - Home (1-paragraph intro + photo).
 - About (longer story).
 - Blog (technical posts).
 - Projects (3–4 deep dives).
 - Contact / CV download.

I2. Blog (write consistently)

Aim for **1 post every 3–4 weeks** during prep months. Topics that work:

- “How I passed the AWS SAA in 6 weeks.”
- “Building a complete observability stack with Prometheus, Grafana, Loki.”
- “From Docker to Kubernetes: my mental model.”
- “5 AWS cost optimization techniques I learned the hard way.”
- “Postmortem: what I learned from [public incident].”
- “Migrating from Jenkins to GitHub Actions: a walkthrough.”

Quality over quantity. Each post should be 1,500–3,000 words, include diagrams, code samples, links.

Distribution:

- Cross-post to **dev.to**, **Hashnode**, **Medium**.
- Share on LinkedIn + Twitter/X.
- Submit to **Hacker News** for big posts.

I3. GitHub strategy

- Pin 6 best repos at top of profile.

- Each repo: README with badges, architecture diagram, setup instructions, example usage.
- Green squares matter less than quality of repos, but try to commit 3+ days/week.

Repos to build:

1. **aws-observability-stack** — Terraform + Helm charts for full Prom/Grafana/Loki/Tempo stack on EKS.
2. **go-microservice-template** — production-ready Go service template (12-factor, healthchecks, structured logging, OTel tracing, Dockerfile, K8s manifests, GitHub Actions CI/CD).
3. **aws-cost-analyzer** — Go CLI that audits AWS account for cost optimization opportunities.
4. **k8s-secure-pipeline** — full secure CI pipeline demonstrating SAST, DAST, container scanning, signing.
5. **terraform-aws-modules-collection** — your reusable Terraform modules.
6. **sre-runbook-templates** — markdown templates for runbooks, post-mortems, ADRs.

I4. Community engagement

- **CNCF Slack** (cloud-native.slack.com) — active SIGs.
- **Kubernetes Slack** (kubernetes.slack.com).
- **r/devops, r/sre, r/kubernetes** on Reddit.
- **AWS Community Builders** program (apply once you have a blog and projects).
- **Local meetups in Dhaka:** Bangladesh DevOps Community, Bangladesh Open Source Network. Attend; eventually speak.

I5. Open source contributions

Pick ONE project to contribute to consistently:

- **Kubernetes** (steep curve but high prestige).
- **Terraform AWS provider** or community modules.
- **OpenTelemetry** SDKs (Go contrib).
- **Argo CD / Argo Rollouts**.
- **Helm charts** in `helm/charts` or `bitnami/charts`.

Start with documentation PRs, then bug fixes, then features. 3–5 merged PRs on a major project is a strong signal.

Part J — Resources Master List

Books (prioritized)

1. **“Site Reliability Engineering”** (Google) — FREE at sre.google/books.
2. **“The Site Reliability Workbook”** (Google) — FREE at sre.google/books.
3. **“Designing Data-Intensive Applications”** (Kleppmann) — \$40 paperback. **THE most important book on this list.**
4. **“Terraform: Up & Running”** (Brikman, 3rd ed.) — \$40.
5. **“Kubernetes Up & Running”** (Burns, Beda, Hightower) — \$40.
6. **“Learning Go”** (Bodner) — \$35.
7. **“System Design Interview Vol 1 & 2”** (Alex Xu) — \$60 combined.
8. **“The Phoenix Project”** + **“The Unicorn Project”** (Gene Kim) — fiction novels about DevOps culture; quick reads, valuable.

Courses

Course	Platform	Cost	Time
AWS SAA-C03 (Stephane Maarek)	Udemy	\$10–15 on sale	30 hrs
AWS SAA practice exams (Tutorials Dojo)	Tutorials Dojo	\$15	10 hrs
CKA (Mumshad Mannambeth)	KodeKloud	\$30/month	35 hrs
HashiCorp Learn Terraform	HashiCorp	FREE	20 hrs
A Tour of Go	go.dev	FREE	5 hrs
Prometheus Certified Associate study	linuxfoundation.org	\$250 cert	30 hrs
FinOps Certified Practitioner	finops.org	\$325	6 hrs

YouTube channels

- **TechWorld with Nana** — best K8s/DevOps tutorials.
- **ByteByteGo** — system design.
- **Bret Fisher Docker and DevOps** — practical Docker/K8s.
- **DevOps Toolkit (Viktor Farcic)** — advanced K8s, GitOps.
- **The Linux Foundation** — KubeCon talks.
- **Honeypot** — engineering culture documentaries.

Newsletters

- **The Pragmatic Engineer** (Gergely Orosz) — industry insights.

- **Last Week in AWS** (Corey Quinn) — AWS news + snark.
- **DevOps Weekly** (Gareth Rushgrove).
- **SRE Weekly**.
- **ByteByteGo Newsletter** (system design).
- **TLDR DevOps** — daily digest.

Job platforms (consolidated)

Singapore:

- LinkedIn (primary)
- NodeFlair (nodeflair.com)
- MyCareersFuture (mycareersfuture.gov.sg)
- Glints (glints.com)
- Glassdoor SG

Japan:

- TokyoDev (tokyodev.com) — **start here**
- Japan Dev (japan-dev.com)
- LinkedIn
- Wantedly (wantedly.com)
- Bizreach (bizreach.jp)
- CareerCross (careercross.com)

Malaysia:

- JobStreet (jobstreet.com.my)
- LinkedIn
- Hiredly (hiredly.com)

Thailand:

- LinkedIn
- Agoda careers (careers.agoda.com)
- JobsDB
- Glassdoor TH

Global (relevant to APAC):

- LinkedIn
- Levels.fyi (for compensation research)
- Glassdoor

Practice platforms

- **LeetCode** (leetcode.com) — coding practice. Premium (\$35/month or \$159/year) unlocks company-specific filters.
- **NeetCode** (neetcode.io) — curated free LeetCode roadmap.
- **Pramp** (pramp.com) — FREE peer mock interviews.

- **interviewing.io** (\$200–400/session) — anonymous mocks with FAANG engineers.
- **Hello Interview** (hellointerview.com) — system design mocks + walk-throughs.
- **Exponent** (tryexponent.com) — behavioral + system design.
- **killer.sh** — CKA exam simulator (free attempts come with CKA registration).

Cost-conscious budget (Bangladesh-friendly)

Essential spend (~\$300 over 6 months):

- AWS SAA exam: \$150.
- CKA exam: \$395 (often \$245 with frequent CNCF discounts).
- Tutorials Dojo SAA practice: \$15.
- LeetCode Premium (3 months): \$35.
- 1 mock interview on interviewing.io: \$200.
- 1 book (DDIA): \$40.

Free alternatives (use these first):

- Andrew Ng courses (audit free on Coursera).
- All Google SRE books.
- All HashiCorp Learn tutorials.
- TechWorld with Nana YouTube.
- NeetCode roadmap.
- Pramp mocks.

Final Words

You're starting from a stronger position than you might realize. Junior SRE with Docker + K8s + CI/CD + some AWS is a **solid foundation** — most candidates apply with less. What separates you from offer letters is:

1. **Depth in 2–3 areas** (not breadth in 10) — pick AWS + K8s + observability and go deep.
2. **A visible body of work** — blog posts + GitHub repos + maybe a conference talk.
3. **Interview preparation** — system design + behavioral, not just LeetCode.
4. **Active networking** — referrals beat cold applications 4:1.
5. **Country-specific tailoring** — TokyoDev for Japan, JobStreet for Malaysia, etc.

Realistic outcome at 15 hrs/week for 6 months: 1–3 offers from SG/JP/MY/TH mid-tier tech companies at SGD 6,500–9,000/month, ¥7M–10M/year, RM12,000–17,000/month, or THB 1.8M–2.4M/year.

One last principle: Don't optimize for the highest salary on day one. Optimize for the **fastest-learning environment** — Grab, Mercari, Agoda all have strong engineering cultures where 2 years there will accelerate your career more than 4 years at a less ambitious place. Comp will follow.

Good luck. Build in public. Ship often. The path is long but every week of focused effort compounds.

Quick reference: Weekly checklist (print this)

- ☐ 5 LeetCode problems solved
- ☐ 1 system design problem walked through
- ☐ 5 hours technical study (courses/books)
- ☐ 4 hours hands-on project work
- ☐ 5 job applications submitted
- ☐ 3 LinkedIn connection requests sent (to engineers at target companies)
- ☐ 1 thoughtful comment on a target company's engineering blog post
- ☐ 1 mock interview (peer or paid) every 2 weeks
- ☐ 1 GitHub commit per day (3 days minimum)
- ☐ 1 blog post per month